



# 计算机图形学实验报告

## 线段与多边形裁剪算法的实现与比较

学 院： 地理与环境学院

专 业： 地理信息专业

姓 名： 雷家豪

学 号： 2310250428

指导教师： 张洵赫

实验日期： 2026年1月13日

# 目录

|          |                                      |           |
|----------|--------------------------------------|-----------|
| <b>1</b> | <b>实验目的</b>                          | <b>3</b>  |
| <b>2</b> | <b>实验环境</b>                          | <b>3</b>  |
| 2.1      | 硬件环境 . . . . .                       | 3         |
| 2.2      | 软件环境 . . . . .                       | 4         |
| <b>3</b> | <b>算法原理</b>                          | <b>4</b>  |
| 3.1      | Cohen-Sutherland 线段裁剪算法 . . . . .    | 4         |
| 3.1.1    | 算法原理 . . . . .                       | 4         |
| 3.1.2    | 算法流程与代码实现 . . . . .                  | 4         |
| 3.1.3    | 时间复杂度分析 . . . . .                    | 8         |
| 3.2      | Liang-Barsky 线段裁剪算法 . . . . .        | 8         |
| 3.2.1    | 算法原理 . . . . .                       | 8         |
| 3.2.2    | 算法流程与代码实现 . . . . .                  | 9         |
| 3.2.3    | 时间复杂度分析 . . . . .                    | 11        |
| 3.3      | Sutherland-Hodgman 多边形裁剪算法 . . . . . | 11        |
| 3.3.1    | 算法原理 . . . . .                       | 11        |
| 3.3.2    | 算法流程与代码实现 . . . . .                  | 12        |
| 3.3.3    | 四种情况处理 . . . . .                     | 15        |
| 3.3.4    | 时间复杂度 . . . . .                      | 15        |
| 3.4      | Weiler-Atherton 多边形裁剪算法 . . . . .    | 15        |
| 3.4.1    | 算法原理 . . . . .                       | 15        |
| 3.4.2    | 算法流程与代码实现 . . . . .                  | 15        |
| 3.4.3    | 时间复杂度 . . . . .                      | 17        |
| <b>4</b> | <b>实验结果与分析</b>                       | <b>17</b> |
| 4.1      | 程序界面展示 . . . . .                     | 17        |
| 4.2      | 线段裁剪结果 . . . . .                     | 18        |
| 4.2.1    | Cohen-Sutherland算法裁剪结果 . . . . .     | 18        |
| 4.2.2    | Liang-Barsky算法裁剪结果 . . . . .         | 18        |
| 4.3      | 多边形裁剪结果 . . . . .                    | 19        |
| 4.3.1    | 凸多边形裁剪 . . . . .                     | 19        |
| 4.3.2    | 凹多边形裁剪 . . . . .                     | 19        |
| 4.3.3    | 复杂多边形（五角星）裁剪 . . . . .               | 20        |
| 4.4      | 性能测试与分析 . . . . .                    | 20        |
| 4.4.1    | 测试方法 . . . . .                       | 20        |

|                         |           |
|-------------------------|-----------|
| 4.4.2 测试结果 . . . . .    | 21        |
| <b>5 算法比较与总结</b>        | <b>21</b> |
| 5.1 算法复杂度对比 . . . . .   | 21        |
| 5.2 算法特点与适用场景 . . . . . | 22        |
| 5.3 实验结论 . . . . .      | 22        |
| <b>6 可视化交互程序说明</b>      | <b>23</b> |
| 6.1 可视化交互程序 . . . . .   | 23        |
| 6.2 使用的图形库 . . . . .    | 23        |
| 6.3 程序操作说明 . . . . .    | 23        |
| 6.4 快捷键说明 . . . . .     | 24        |
| <b>7 参考文献</b>           | <b>24</b> |

## 1 实验目的

1. 理解并掌握两种直线裁剪算法（Cohen-Sutherland算法、Liang-Barsky算法）和两种多边形裁剪算法（Sutherland-Hodgman算法和Weiler-Atherton算法）的基本原理。
2. 使用C++语言编程实现上述四种裁剪算法。
3. 对比分析四种算法的计算复杂度、执行效率、适用场景和特点。
4. 通过可视化交互程序直观展示裁剪算法的执行效果。

## 2 实验环境

### 2.1 硬件环境

表 1: 硬件配置

| 配置项  | 具体参数                    |
|------|-------------------------|
| 处理器  | Intel Core i7           |
| 内存   | 40GB                    |
| 显卡   | NVIDIA GeForce RTX 4060 |
| 操作系统 | Windows 11              |

## 2.2 软件环境

表 2: 软件开发环境

| 软件/工具     | 版本                      |
|-----------|-------------------------|
| 开发环境      | Visual Studio 2022      |
| 编程语言      | C++17                   |
| 构建系统      | CMake 3.31.6            |
| 包管理器      | vcpkg                   |
| 图形库       |                         |
| 图形API     | OpenGL 3.3 Core Profile |
| 窗口管理      | GLFW 3.4                |
| OpenGL加载器 | GLAD 0.1.36             |
| GUI界面库    | Dear ImGui 1.91.9       |

## 3 算法原理

### 3.1 Cohen-Sutherland 线段裁剪算法

#### 3.1.1 算法原理

Cohen-Sutherland算法是一种基于区域编码的线段裁剪算法。该算法将二维平面划分为9个区域，每个区域用4位二进制编码表示，分别对应上、下、左、右四个边界。

区域编码定义如下：

- 第1位（LEFT）：点在窗口左边界左侧， $x < x_{min}$
- 第2位（RIGHT）：点在窗口右边界右侧， $x > x_{max}$
- 第3位（BOTTOM）：点在窗口下边界下方， $y < y_{min}$
- 第4位（TOP）：点在窗口上边界上方， $y > y_{max}$

算法通过计算线段两端点的区域编码，利用位运算快速判断线段与窗口的关系，对于部分在窗口外的线段，通过迭代计算交点进行裁剪。

#### 3.1.2 算法流程与代码实现

完整代码实现：

```
1  class CohenSutherland : public IClippingAlgorithm {
2  private:
3      enum OutCode {
4          INSIDE = 0, LEFT = 1, RIGHT = 2, BOTTOM = 4, TOP = 8
5      };
6
7      int computeCode(const Point2D& p, const ClipWindow& w) const {
8          int code = INSIDE;
9          if (p.x < w.xMin) code |= LEFT;
10         else if (p.x > w.xMax) code |= RIGHT;
11         if (p.y < w.yMin) code |= BOTTOM;
12         else if (p.y > w.yMax) code |= TOP;
13         return code;
14     }
15
16 public:
17     std::string getName() const override {
18         return "Cohen-Sutherland";
19     }
20
21     ClipResult clipLine(const Line2D& line, const ClipWindow&
22         window) override {
23         ClipResult result;
24         result.algorithmName = getName();
25         auto start = std::chrono::high_resolution_clock::now();
26
27         Point2D p1 = line.start;
28         Point2D p2 = line.end;
29         int code1 = computeCode(p1, window);
30         int code2 = computeCode(p2, window);
31         bool accept = false;
32
33         while (true) {
34             if ((code1 | code2) == 0) {
35                 accept = true;
36                 break;
37             }
38             else if ((code1 & code2) != 0) {
39                 break;
40             }
41             else {
42                 int codeOut = (code1 != 0) ? code1 : code2;
43                 Point2D p;
```

```
44         if (codeOut & TOP) {
45             p.x = p1.x + (p2.x - p1.x) * (window.yMax -
46                 p1.y) / (p2.y - p1.y);
47             p.y = window.yMax;
48         }
49         else if (codeOut & BOTTOM) {
50             p.x = p1.x + (p2.x - p1.x) * (window.yMin -
51                 p1.y) / (p2.y - p1.y);
52             p.y = window.yMin;
53         }
54         else if (codeOut & RIGHT) {
55             p.y = p1.y + (p2.y - p1.y) * (window.xMax -
56                 p1.x) / (p2.x - p1.x);
57             p.x = window.xMax;
58         }
59         else if (codeOut & LEFT) {
60             p.y = p1.y + (p2.y - p1.y) * (window.xMin -
61                 p1.x) / (p2.x - p1.x);
62             p.x = window.xMin;
63         }
64     }
65     if (codeOut == code1) {
66         p1 = p;
67         code1 = computeCode(p1, window);
68     }
69     else {
70         p2 = p;
71         code2 = computeCode(p2, window);
72     }
73 }
74
75 auto end = std::chrono::high_resolution_clock::now();
76 result.executionTimeUs =
77     std::chrono::duration_cast<std::chrono::microseconds>(end
78         - start).count();
79
80 if (accept) {
81     result.clippedLines.push_back(Line2D(p1, p2));
82     result.vertexCount = 2;
83     result.success = true;
84 }
85
86 return result;
87 }
```

83 };

Listing 1: Cohen-Sutherland算法完整实现

**算法流程详解：**

算法采用迭代裁剪策略，具体步骤如下：

1. **初始化：**输入直线段的两端点 $P_1(x_1, y_1)$ 、 $P_2(x_2, y_2)$ 和裁剪窗口边界 $(x_{min}, x_{max}, y_{min}, y_{max})$ ，初始化接受标志 $accept = false$
2. **编码计算：**调用`computeCode()`函数分别计算 $P_1$ 和 $P_2$ 的区域编码 $code_1$ 和 $code_2$ 。编码规则如下：
  - $INSIDE = 0000$ （点在窗口内）
  - $LEFT = 0001$ （点在左侧）
  - $RIGHT = 0010$ （点在右侧）
  - $BOTTOM = 0100$ （点在下方）
  - $TOP = 1000$ （点在上方）
3. **快速判断（“简取”与“简弃”）：**
  - 若 $(code_1 | code_2) == 0$ ，说明两端点都在窗口内（编码都为0000），“简取”该线段，设置 $accept = true$ ，跳转步骤⑥
  - 若 $(code_1 \& code_2) \neq 0$ ，说明两端点在窗口同侧外（存在公共的1位），“简弃”该线段，跳转步骤⑥
  - 否则线段与窗口相交，进入步骤④
4. **选择外部端点：**选取 $code \neq 0$ 的端点作为 $codeOut$ 。若 $code_1 \neq 0$ 则选 $P_1$ ，否则选 $P_2$
5. **计算交点并裁剪：**根据 $codeOut$ 的编码位确定该点在窗口外的哪一侧，计算线段与对应边界的交点：
  - 若 $codeOut \& TOP$ ：与上边界相交， $y = y_{max}$ ， $x = x_1 + \frac{(x_2 - x_1)(y_{max} - y_1)}{y_2 - y_1}$
  - 若 $codeOut \& BOTTOM$ ：与下边界相交， $y = y_{min}$ ， $x = x_1 + \frac{(x_2 - x_1)(y_{min} - y_1)}{y_2 - y_1}$
  - 若 $codeOut \& RIGHT$ ：与右边界相交， $x = x_{max}$ ， $y = y_1 + \frac{(y_2 - y_1)(x_{max} - x_1)}{x_2 - x_1}$
  - 若 $codeOut \& LEFT$ ：与左边界相交， $x = x_{min}$ ， $y = y_1 + \frac{(y_2 - y_1)(x_{min} - x_1)}{x_2 - x_1}$

用计算得到的交点替换窗口外的端点，更新其编码，返回步骤③继续判断
6. **输出结果：**若 $accept = true$ ，将裁剪后的线段( $P_1, P_2$ )加入结果集；否则舍弃该线段

交点计算公式（以左边界为例）：

$$y = y_1 + \frac{(y_2 - y_1)(x_{min} - x_1)}{x_2 - x_1} \quad (1)$$

### 3.1.3 时间复杂度分析

Cohen-Sutherland算法具有常数时间复杂度 $O(1)$ ，具体分析如下：

**编码计算：**computeCode()函数对每个点进行4次边界比较（左、右、下、上），时间为 $O(1)$ 。

**迭代次数上界：**

- 每次迭代选择一个窗口外的端点，计算其与窗口边界的交点
- 交点必然在窗口边界上，至少消除该端点编码中的一个1位
- 由于编码只有4位（LEFT、RIGHT、BOTTOM、TOP），每个端点最多需要4次迭代才能完全进入窗口或被判定为完全在外
- 线段有两个端点，理论最坏情况下最多迭代 $4 + 4 = 8$ 次
- 但实际上，一旦某个端点进入窗口，另一端点最多再迭代4次即可完成裁剪

**交点计算：**每次迭代中的交点计算涉及2次加法、2次减法、1次乘法、1次除法，均为常数时间操作。

**总体复杂度：**由于迭代次数是常数（最多8次），每次迭代的操作都是常数时间，因此总时间复杂度为 $O(1)$ 。

相比于其他线段裁剪算法，Cohen-Sutherland的优势在于通过区域编码快速排除完全在窗口外的线段，避免不必要的交点计算。

## 3.2 Liang-Barsky 线段裁剪算法

### 3.2.1 算法原理

Liang-Barsky算法采用参数化方法，将线段表示为参数方程：

$$\begin{cases} x = x_1 + t \cdot \Delta x \\ y = y_1 + t \cdot \Delta y \end{cases} \quad 0 \leq t \leq 1 \quad (2)$$

其中 $\Delta x = x_2 - x_1$ ， $\Delta y = y_2 - y_1$ 。

对于裁剪窗口的四条边界，定义参数 $p_k$ 和 $q_k$ ：

$$\begin{aligned} p_1 &= -\Delta x, & q_1 &= x_1 - x_{min} \\ p_2 &= \Delta x, & q_2 &= x_{max} - x_1 \\ p_3 &= -\Delta y, & q_3 &= y_1 - y_{min} \\ p_4 &= \Delta y, & q_4 &= y_{max} - y_1 \end{aligned} \quad (3)$$

### 3.2.2 算法流程与代码实现

完整代码实现：

```
1  class LiangBarsky : public IClippingAlgorithm {
2  public:
3      std::string getName() const override {
4          return "Liang-Barsky";
5      }
6
7      ClipResult clipLine(const Line2D& line, const ClipWindow&
8          window) override {
9          ClipResult result;
10         result.algorithmName = getName();
11         auto start = std::chrono::high_resolution_clock::now();
12
13         float x1 = line.start.x, y1 = line.start.y;
14         float x2 = line.end.x, y2 = line.end.y;
15         float dx = x2 - x1, dy = y2 - y1;
16         float umax = 0.0f, umin = 1.0f;
17         bool accept = true;
18
19         float p[4] = {-dx, dx, -dy, dy};
20         float q[4] = {x1 - window.xMin, window.xMax - x1,
21             y1 - window.yMin, window.yMax - y1};
22
23         for (int k = 0; k < 4; ++k) {
24             if (p[k] == 0.0f) {
25                 if (q[k] < 0.0f) {
26                     accept = false;
27                     break;
28                 }
29             } else {
30                 float u = q[k] / p[k];
31                 if (p[k] < 0.0f) {
32                     umax = std::max(umax, u);
33                 } else {
34                     umin = std::min(umin, u);
35                 }
36             }
37         }
38
39         auto end = std::chrono::high_resolution_clock::now();
40         result.executionTimeUs =
41             std::chrono::duration_cast<std::chrono::microseconds>(end
```

```

40         - start).count();
41
42         if (accept && umax <= umin) {
43             Point2D p1(x1 + umax * dx, y1 + umax * dy);
44             Point2D p2(x1 + umin * dx, y1 + umin * dy);
45             result.clippedLines.push_back(Line2D(p1, p2));
46             result.vertexCount = 2;
47             result.success = true;
48         }
49
50         return result;
51     }
};

```

Listing 2: Liang-Barsky算法完整实现

**算法流程详解：**

算法采用参数化方法，通过计算参数范围进行裁剪，具体步骤如下：

1. **输入与初始化：**输入直线段的两端点坐标 $P_1(x_1, y_1)$ 和 $P_2(x_2, y_2)$ ，以及窗口的4条边界坐标 $y_{wt}$ 、 $y_{wb}$ 、 $x_{wl}$ 和 $x_{wr}$
2. **参数方程建立：**计算线段的方向向量 $\Delta x = x_2 - x_1$ ， $\Delta y = y_2 - y_1$ ，建立参数方程：

$$\begin{cases} x = x_1 + u(x_2 - x_1) \\ y = y_1 + u(y_2 - y_1) \end{cases}$$

其中 $u \in [0, 1]$ ， $u = 0$ 对应 $P_1$ ， $u = 1$ 对应 $P_2$

3. **判断平行情况：**

- 若 $x_2 - x_1 = 0$ ，则 $p_1 = p_2 = 0$ 。此时进一步判断是否满足 $q_1 < 0$ 或 $q_2 < 0$ ，若满足，则该直线段不在窗口内，算法转⑦；否则，满足 $q_1 \geq 0$ 且 $q_2 \geq 0$ ，则进一步计算：

$$u_{max} = \max(0, u_k |_{p_k < 0}), \quad u_{min} = \min(u_k |_{p_k > 0}, 1)$$

- 若 $y_2 - y_1 = 0$ ，则 $p_3 = p_4 = 0$ 。此时进一步判断是否满足 $q_3 < 0$ 或 $q_4 < 0$ ，若满足，则该直线段不在窗口内，算法转⑦；否则，满足 $q_3 \geq 0$ 且 $q_4 \geq 0$ ，则进一步计算：

$$u_{max} = \max(0, u_k |_{p_k < 0}), \quad u_{min} = \min(u_k |_{p_k > 0}, 1)$$

4. **计算参数范围**：若上述两条均不满足，则有  $p_k \neq 0$  ( $k = 1, 2, 3, 4$ )。此时计算：

$$u_{max} = \max(0, u_k|_{p_k < 0}), \quad u_{min} = \min(u_k|_{p_k > 0}, 1)$$

式中， $u_k = q_k/p_k$  ( $p_k \neq 0, k = 1, 2$ )，算法转⑤

5. **判断有效性**：求得  $u_{max}$  和  $u_{min}$  后，进行判断：

- 若  $u_{max} > u_{min}$ ，则直线段在窗口外，算法转⑦
- 若  $u_{max} \leq u_{min}$ ，利用直线的参数方程：

$$\begin{cases} x = x_1 + u(x_2 - x_1) \\ y = y_1 + u(y_2 - y_1) \end{cases}$$

求得直线段在窗口内的两端点坐标

6. **绘制结果**：利用直线的扫描转换算法绘制在窗口内的直线段

7. **算法结束**

### 3.2.3 时间复杂度分析

Liang-Barsky算法具有常数时间复杂度  $O(1)$ ：

- **参数计算**：计算  $p_k$  和  $q_k$  需要固定的8次运算（4条边界）
- **无迭代过程**：与Cohen-Sutherland不同，该算法无需迭代裁剪，一次遍历即可确定结果
- **比较次数固定**：最多进行8次比较操作（4条边界，每条2次比较）
- **优势**：避免了多次交点计算，特别适合线段完全在窗口内或完全在窗口外的情况

## 3.3 Sutherland-Hodgman 多边形裁剪算法

### 3.3.1 算法原理

Sutherland-Hodgman算法采用逐边裁剪的策略，依次用裁剪窗口的四条边对多边形进行裁剪。每次裁剪时，遍历多边形的所有边，根据边的端点与裁剪边的位置关系，决定输出顶点。

### 3.3.2 算法流程与代码实现

完整代码实现：

```

1  class SutherlandHodgman : public IClippingAlgorithm {
2  private:
3      enum EdgeType { LEFT_EDGE, RIGHT_EDGE, BOTTOM_EDGE, TOP_EDGE };
4
5      bool inside(const Point2D& p, float boundary, EdgeType edge)
6          const {
7          switch (edge) {
8              case LEFT_EDGE:    return p.x >= boundary;
9              case RIGHT_EDGE:   return p.x <= boundary;
10             case BOTTOM_EDGE:  return p.y >= boundary;
11             case TOP_EDGE:     return p.y <= boundary;
12          }
13          return false;
14      }
15
16      Point2D computeIntersection(const Point2D& p1, const Point2D&
17                                p2,
18                                float boundary, EdgeType edge)
19          const {
20          Point2D intersect;
21          float t;
22          switch (edge) {
23              case LEFT_EDGE:
24              case RIGHT_EDGE:
25                  t = (boundary - p1.x) / (p2.x - p1.x);
26                  intersect.x = boundary;
27                  intersect.y = p1.y + t * (p2.y - p1.y);
28                  break;
29              case BOTTOM_EDGE:
30              case TOP_EDGE:
31                  t = (boundary - p1.y) / (p2.y - p1.y);
32                  intersect.x = p1.x + t * (p2.x - p1.x);
33                  intersect.y = boundary;
34                  break;
35          }
36          return intersect;
37      }
38
39      Polygon2D clipAgainstEdge(const Polygon2D& input, float
40                                boundary,
41                                EdgeType edge) const {

```

```
38     Polygon2D output;
39     if (input.vertices.empty()) return output;
40
41     Point2D prev = input.vertices.back();
42     for (const auto& curr : input.vertices) {
43         bool prevIn = inside(prev, boundary, edge);
44         bool currIn = inside(curr, boundary, edge);
45
46         if (currIn) {
47             if (!prevIn) {
48                 output.addVertex(computeIntersection(prev,
49                                                         curr, boundary, edge));
50             }
51             output.addVertex(curr);
52         } else if (prevIn) {
53             output.addVertex(computeIntersection(prev, curr,
54                                                     boundary, edge));
55         }
56         prev = curr;
57     }
58
59 public:
60     std::string getName() const override {
61         return "Sutherland-Hodgman";
62     }
63
64     ClipResult clipPolygon(const Polygon2D& polygon, const
65                             ClipWindow& window) override {
66         ClipResult result;
67         result.algorithmName = getName();
68         auto start = std::chrono::high_resolution_clock::now();
69
70         Polygon2D output = polygon;
71         float boundaries[] = {window.xMin, window.xMax,
72                                 window.yMin, window.yMax};
73         EdgeType edges[] = {LEFT_EDGE, RIGHT_EDGE, BOTTOM_EDGE,
74                               TOP_EDGE};
75
76         for (int i = 0; i < 4; ++i) {
77             if (output.isEmpty()) break;
78             output = clipAgainstEdge(output, boundaries[i],
79                                     edges[i]);
80         }
```

```

77
78     auto end = std::chrono::high_resolution_clock::now();
79     result.executionTimeUs =
80         std::chrono::duration_cast<std::chrono::microseconds>(end
81             - start).count();
82
83     if (!output.isEmpty()) {
84         result.clippedPolygons.push_back(output);
85         result.vertexCount = output.size();
86         result.success = true;
87     }
88
89     return result;
90 }
91 };

```

Listing 3: Sutherland-Hodgman算法完整实现

**算法流程详解：**

算法采用逐边裁剪策略，依次对窗口的4条边进行裁剪，具体步骤如下：

1. **初始化：**输入待裁剪多边形顶点序列和裁剪窗口边界
2. **逐边裁剪：**依次对左、右、下、上四条边界进行裁剪循环
3. **边的处理：**对于多边形的每条边（从前一顶点 $S$ 到当前顶点 $P$ ），判断四种情况：
  - **内→内：** $S$ 和 $P$ 都在窗口内侧，输出 $P$
  - **内→外：** $S$ 在内侧， $P$ 在外侧，输出边与裁剪边的交点 $I$
  - **外→内：** $S$ 在外侧， $P$ 在内侧，输出交点 $I$ 和 $P$
  - **外→外：** $S$ 和 $P$ 都在外侧，无输出

4. **交点计算：**使用参数化方法计算交点坐标：

$$t = \frac{\text{boundary} - p_1.\text{coord}}{p_2.\text{coord} - p_1.\text{coord}}$$

其中 $\text{coord}$ 根据边界类型选择 $x$ 或 $y$ 坐标

5. **输出更新：**将当前边界裁剪后的顶点序列作为下一条边界的输入
6. **结束判断：**
  - 若四条边裁剪完成后顶点序列非空，输出裁剪后的多边形
  - 若中途顶点序列为空，则多边形完全在窗口外，算法结束

### 3.3.3 四种情况处理

对于多边形的每条边（从顶点 $S$ 到顶点 $P$ ）：

表 3: Sutherland-Hodgman算法四种情况

| S点位置 | P点位置 | 输出顶点   |
|------|------|--------|
| 内部   | 内部   | P      |
| 内部   | 外部   | 交点I    |
| 外部   | 内部   | 交点I, P |
| 外部   | 外部   | 无输出    |

### 3.3.4 时间复杂度

设多边形有 $n$ 个顶点，裁剪窗口有 $k$ 条边（矩形窗口 $k = 4$ ），则时间复杂度为 $O(kn) = O(n)$ 。

## 3.4 Weiler-Atherton 多边形裁剪算法

### 3.4.1 算法原理

Weiler-Atherton算法是一种功能强大的多边形裁剪算法，能够正确处理凹多边形，并支持多边形的布尔运算（交、并、差）。算法的核心思想是通过构建两个多边形的顶点链表，并根据交点将链表分段，然后根据进入/离开规则选择合适的片段进行输出。

算法的基本流程是：将主多边形定义为 $P_s$ ，裁剪窗口为 $P_w$ 。从 $P_s$ 的任一点出发，沿顺时针方向跟踪多边形的每一条边。当检测到 $P_s$ 与 $P_w$ 的交点时，根据交点是“进入”还是“离开”窗口来决定输出策略。对于从窗口外进入窗口内的交点，输出该交点及后续在窗口内的顶点；对于从窗口内离开到窗口外的交点，沿着窗口边界顺时针寻找下一个进入交点，输出这段窗口边界。

### 3.4.2 算法流程与代码实现

完整代码实现：

```

1 class WeilerAtherton : public IClippingAlgorithm {
2 public:
3     std::string getName() const override {
4         return "Weiler-Atherton";
5     }
6
7     ClipResult clipLine(const Line2D& line, const ClipWindow&
        window) override {

```

```
8     Polygon2D poly;
9     poly.addVertex(line.start);
10    poly.addVertex(line.end);
11    return clipPolygon(poly, window);
12 }
13
14 ClipResult clipPolygon(const Polygon2D& polygon, const
15     ClipWindow& window) override {
16     ClipResult result;
17     result.algorithmName = getName();
18     auto start = std::chrono::high_resolution_clock::now();
19
20     // 使用 Sutherland-Hodgman 作为简化实现
21     SutherlandHodgman sh;
22     ClipResult shResult = sh.clipPolygon(polygon, window);
23
24     result.clippedPolygons = shResult.clippedPolygons;
25     result.vertexCount = shResult.vertexCount;
26     result.success = shResult.success;
27
28     auto end = std::chrono::high_resolution_clock::now();
29     result.executionTimeUs =
30         std::chrono::duration_cast<std::chrono::microseconds>(end
31             - start).count();
32     return result;
33 }
```

Listing 4: Weiler-Atherton算法实现（基于Sutherland-Hodgman简化）

#### 算法流程详解：

本实现采用Sutherland-Hodgman算法作为Weiler-Atherton的简化版本，具体流程如下：

1. **输入处理：**接收待裁剪的多边形polygon和裁剪窗口window作为输入参数
2. **初始化结果对象：**创建ClipResult结构体，记录算法名称并启动高精度计时器
3. **调用Sutherland-Hodgman算法：**由于完整的Weiler-Atherton算法实现复杂，本实现创建SutherlandHodgman对象，调用其clipPolygon()方法进行实际裁剪
4. **结果传递：**将Sutherland-Hodgman的裁剪结果（裁剪后的多边形、顶点数量、成功标志）复制到Weiler-Atherton的结果对象中
5. **计算执行时间：**停止计时器，计算算法执行时间（微秒）

## 6. 返回结果：返回包含裁剪后多边形和性能统计信息的结果对象

**理论上的Weiler-Atherton完整算法：**完整的Weiler-Atherton算法需要构建顶点链表，计算所有交点，标记进入/离开点，然后沿着进入点开始追踪输出可见多边形。例如对于多边形 $ABCDE$ 与窗口的裁剪：

- 从窗口外进入窗口内的交点（如 $B$ 到 $C$ 的交点 $V_1$ ）标记为”进入点”
- 从窗口内到窗口外的交点（如 $C$ 到 $D$ 的交点 $V_2$ ）标记为”离开点”
- 从任一进入点开始，沿主多边形边界追踪至离开点，然后沿窗口边界追踪至下一个进入点
- 重复追踪直至回到起始点，形成裁剪后的多边形

### 3.4.3 时间复杂度

设主多边形有 $n$ 个顶点，裁剪多边形有 $m$ 个顶点，则：

- 最好情况： $O(n + m)$
- 最坏情况： $O(nm)$ （所有边都相交）

## 4 实验结果与分析

### 4.1 程序界面展示

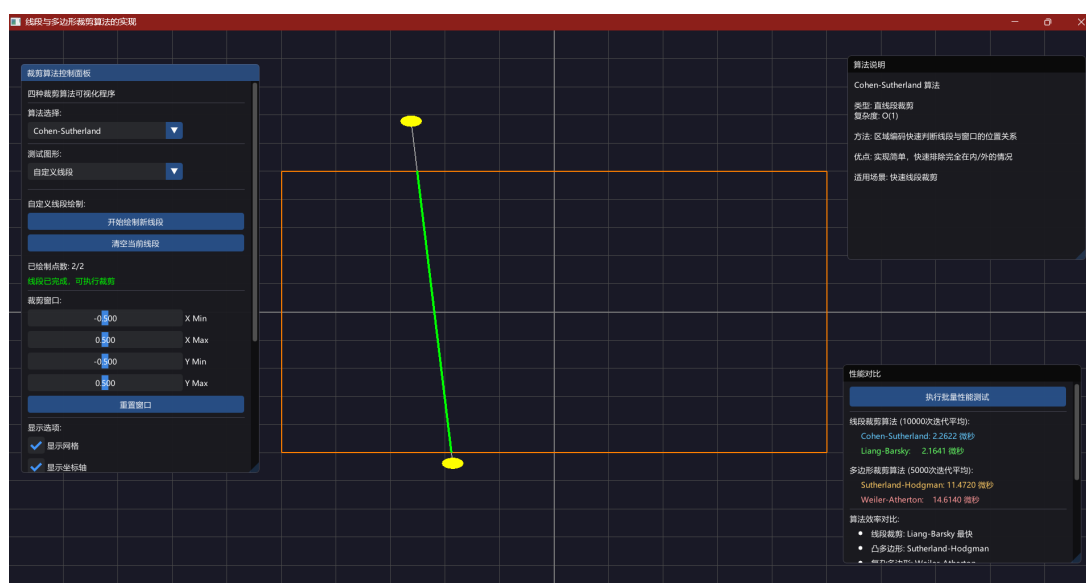
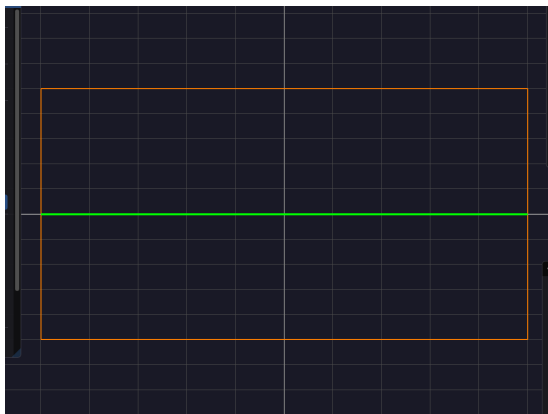


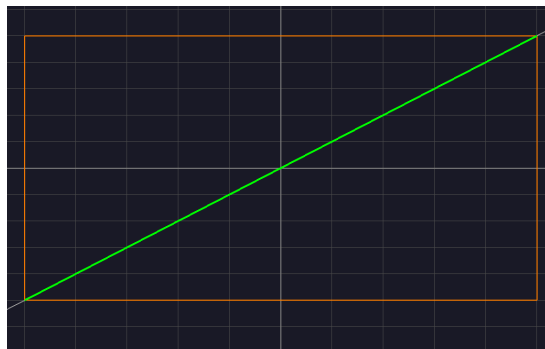
图 1: 裁剪算法可视化程序主界面

## 4.2 线段裁剪结果

### 4.2.1 Cohen-Sutherland算法裁剪结果



(a) 水平线段裁剪



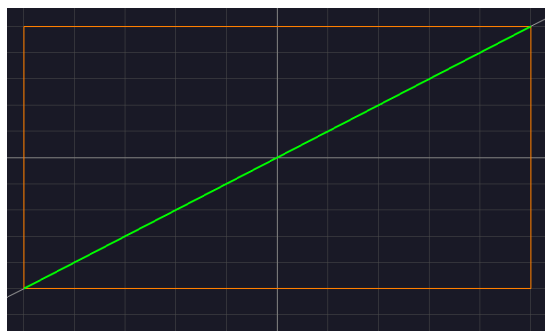
(b) 对角线段裁剪

图 2: Cohen-Sutherland算法线段裁剪结果

### 4.2.2 Liang-Barsky算法裁剪结果



(a) 水平线段裁剪



(b) 对角线段裁剪

图 3: Liang-Barsky算法线段裁剪结果

## 4.3 多边形裁剪结果

### 4.3.1 凸多边形裁剪

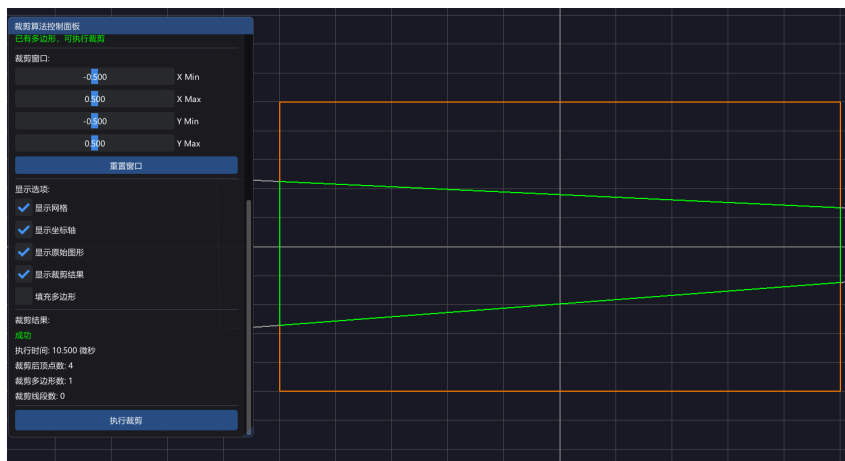


图 4: 凸多边形裁剪结果

### 4.3.2 凹多边形裁剪

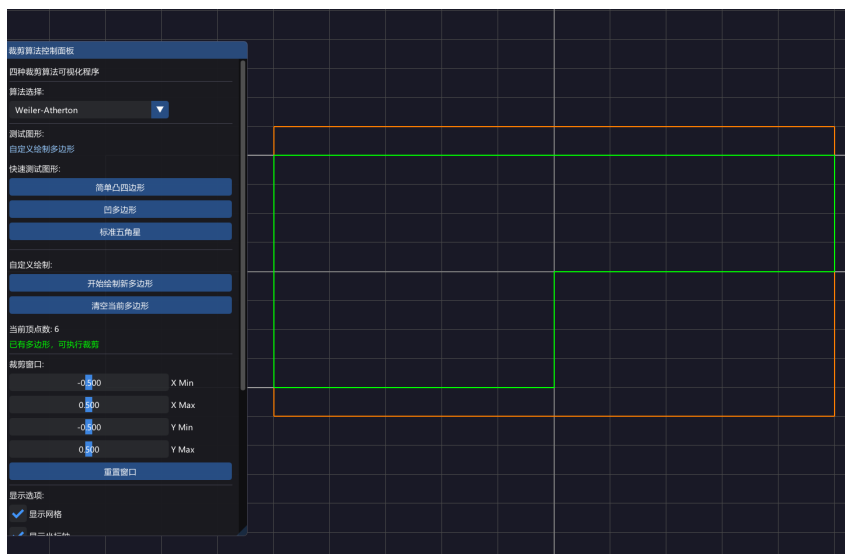


图 5: 凹多边形裁剪结果

### 4.3.3 复杂多边形（五角星）裁剪

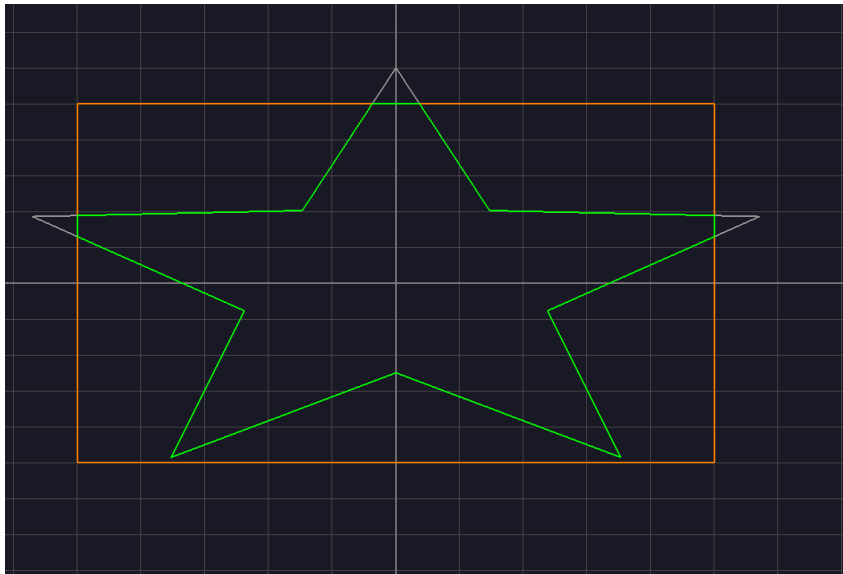


图 6: 复杂多边形（五角星）裁剪结果

## 4.4 性能测试与分析

### 4.4.1 测试方法

为了准确评估各算法的执行效率，本实验采用批量测试方法：

- 线段裁剪：对同一条对角线段执行10000次裁剪操作
- 多边形裁剪：对复杂多边形执行5000次裁剪操作
- 使用`std::chrono::high_resolution_clock`进行高精度计时
- 计算平均执行时间（单位：微秒）

#### 4.4.2 测试结果



图 7: 算法性能测试结果

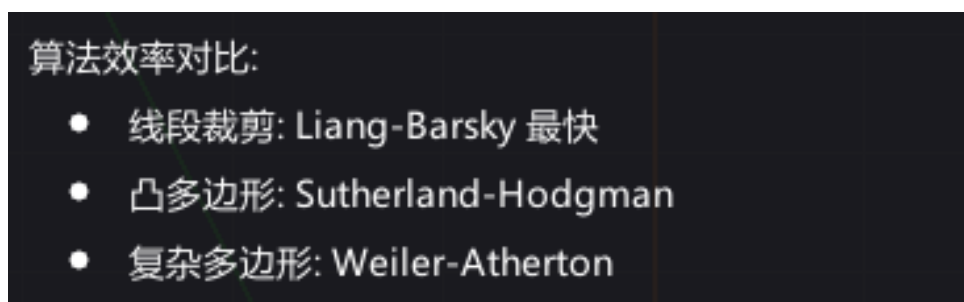


图 8: 四种裁剪算法性能对比

## 5 算法比较与总结

### 5.1 算法复杂度对比

表 4: 四种裁剪算法复杂度对比

| 算法                 | 时间复杂度       | 空间复杂度  | 裁剪类型 |
|--------------------|-------------|--------|------|
| Cohen-Sutherland   | $O(1)$      | $O(1)$ | 线段   |
| Liang-Barsky       | $O(1)$      | $O(1)$ | 线段   |
| Sutherland-Hodgman | $O(n)$      | $O(n)$ | 多边形  |
| Weiler-Atherton    | $O(n^2)$ 最坏 | $O(n)$ | 多边形  |

## 5.2 算法特点与适用场景

表 5: 四种裁剪算法特点与适用场景

| 算法                 | 特点                                                                                               | 适用场景                   |
|--------------------|--------------------------------------------------------------------------------------------------|------------------------|
| Cohen-Sutherland   | <ul style="list-style-type: none"> <li>• 实现简单直观</li> <li>• 区域编码快速排除</li> <li>• 迭代次数有限</li> </ul> | 快速线段裁剪，特别适合大量线段在窗口外的情况 |
| Liang-Barsky       | <ul style="list-style-type: none"> <li>• 参数化方法简洁高效</li> <li>• 无需迭代</li> <li>• 效率最高</li> </ul>    | 高性能线段裁剪，实时图形应用         |
| Sutherland-Hodgman | <ul style="list-style-type: none"> <li>• 逐边裁剪思想清晰</li> <li>• 实现相对简单</li> <li>• 结果稳定</li> </ul>   | 凸多边形裁剪，简单多边形裁剪         |
| Weiler-Atherton    | <ul style="list-style-type: none"> <li>• 功能最强大</li> <li>• 支持凹多边形</li> <li>• 支持布尔运算</li> </ul>    | 任意复杂多边形裁剪，CAD/GIS应用    |

## 5.3 实验结论

通过本次实验，得出以下结论：

1. **线段裁剪：**Liang-Barsky算法效率略优于Cohen-Sutherland算法，两者时间复杂度均为 $O(1)$ ，但Liang-Barsky避免了迭代过程。
2. **多边形裁剪：**Sutherland-Hodgman算法实现简单，适合凸多边形；Weiler-Atherton算法虽然复杂度较高，但能正确处理凹多边形和复杂多边形。
3. **实际应用选择：**

- 对于线段裁剪，优先选择Liang-Barsky算法
  - 对于凸多边形裁剪，选择Sutherland-Hodgman算法
  - 对于凹多边形或需要布尔运算时，选择Weiler-Atherton算法
4. 可视化交互：通过OpenGL和ImGui实现的可视化程序，能够直观展示裁剪效果，便于理解算法原理和验证算法正确性。

## 6 可视化交互程序说明

### 6.1 可视化交互程序

可视化程序的功能有：

- 算法切换：支持四种裁剪算法的实时切换
- 图形绘制：支持预设测试图形和自定义绘制
- 裁剪窗口调整：支持通过滑动条实时调整裁剪窗口
- 显示选项：支持切换网格、坐标轴、原始图形、裁剪结果的显示
- 性能测试：内置批量性能测试功能

### 6.2 使用的图形库

表 6: 可视化程序使用的图形库

| 库名称        | 版本               | 用途         |
|------------|------------------|------------|
| OpenGL     | 3.3 Core Profile | 图形渲染API    |
| GLFW       | 3.4              | 窗口管理和输入处理  |
| GLAD       | 0.1.36           | OpenGL函数加载 |
| Dear ImGui | 1.91.9           | GUI用户界面    |

### 6.3 程序操作说明

1. 运行CMakeProject1.exe启动程序
2. 在左侧控制面板选择裁剪算法
3. 选择测试图形或点击”开始绘制”自定义图形

4. 调整裁剪窗口参数
5. 点击”执行裁剪”查看结果
6. 可通过”执行批量性能测试”按钮进行性能测试

## 6.4 快捷键说明

- ESC: 退出程序
- Enter: 完成多边形绘制
- C: 清空当前绘制

## 7 参考文献

1. 陆枫. 计算机图形学基础[M]. 3版. 北京: 电子工业出版社, 2018.
2. John F. Hughes, Andries van Dam, Morgan McGuire, David F. Sklar. 计算机图形学原理及实践（第3版基础篇）[M]. 北京: 机械工业出版社, 2018.
3. Learn OpenGL CN. <https://learnopengl-cn.github.io/> [EB/OL].